

Benchmarking Retrieval-Augmented Generation Pipelines

A Comparative Study of Chunking Strategies, Embedding Models, and Large Language Models

Project Guide: [Guide Name, Designation]

Team (Group No. 125): Sumit Chauhan • Ankit Yadav • Shikhar Saxena • Tejas Kumar Sahu • Tanuj Vatsa

Introduction

- **The problem** LLMs lack access to private or current knowledge and hallucinate on specialized topics
- **RAG** retrieves relevant passages from an external knowledge base and grounds the answer in them
- **Pipeline** chunk → embed → index → retrieve top-k → generate → evaluate
- **Why chunking matters** it defines the retrieval unit; bad boundaries split ideas or add noise
- **This project** benchmarks 5 chunking strategies with a fixed embedding model and LLM

Existing Work and Limitations

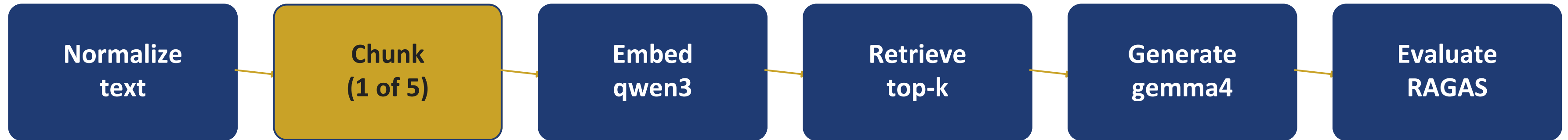
Existing work

- RAG introduced as parametric + non-parametric memory [1]
- Surveys describe RAG as a multi-stage pipeline [2]
- RAGAS gives automated retrieval and generation metrics [3]
- EvaRAG benchmarks retrievers, indexes and metrics [4]

Limitations

- Settings tuned by heuristic trial-and-error
- Several components changed together, so causes are unclear
- Chunking rarely isolated as a variable
- Latency often ignored next to quality
- Often built on closed-source APIs

Proposed Work and Methodology



- **Controlled design** only chunking varies; embedding, LLM, questions, prompt, temperature and top-p are fixed
- **Factorial matrix** $5 \times 1 \times 1 = 5$ configurations (C01–C05)
- **Metrics** Context Recall, Context Precision, Faithfulness, Answer Relevancy, end-to-end latency
- **Selection** composite score $S(c) = (\text{Recall} + \text{Precision} + \text{Faithfulness}) / 3$, with Pareto analysis

Novelty of the Project

Isolated variable

- Chunking is the only factor changed, so effects are attributable

Open-weight stack

- qwen3-embedding with gemma4:31b-cloud instead of legacy closed APIs

Five strategies

- Includes semantic and LLM-driven agentic chunking

Quality + speed

- Composite score and Pareto frontier weigh accuracy against latency

Reusable engine

- Configuration-driven, modular benchmark with a reproducibility checklist

Real-Time Usage

- **Enterprise document Q&A** policies, manuals and internal wikis
- **Customer support assistants** grounded answers from product documentation
- **Legal, medical and technical search** precise retrieval from dense, domain documents
- **Academic and research assistants** question answering over papers and PDFs
- **Production tuning** choose a chunker by latency budget, e.g. fixed-size for speed, agentic for precision

Hardware and Software Requirements

Software

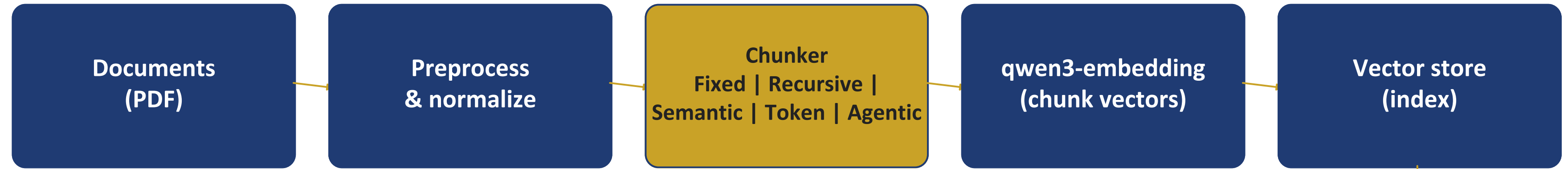
- Python 3 environment
- Ollama runtime: qwen3-embedding:latest and gemma4:31b-cloud
- RAGAS evaluation framework
- Local vector store with cosine similarity
- PDF text extraction and tokenizer libraries
- Plotting library for charts

Hardware

- Laptop or desktop PC (multi-core CPU)
- 16 GB RAM recommended
- Stable internet (cloud-hosted LLM)
- Few GB free disk for models and vector index
- GPU optional for local embedding speed-up

Overall System Architecture

Indexing phase (offline)



Query phase (online)



RAGAS evaluation (Recall, Precision, Faithfulness, Relevancy) + latency logger

Literature Review

Gap: no controlled study isolating chunking strategy on an open-weight embedding + LLM stack.

Module Description

1. Ingestion

- Parse PDF, clean Unicode and whitespace, produce uniform text

2. Chunk generation

- Run one of five chunkers chosen by configuration

3. Embedding & indexing

- Embed chunks with qwen3-embedding; store vectors with metadata

4. Retrieval

- Embed query; cosine similarity search for top-k chunks

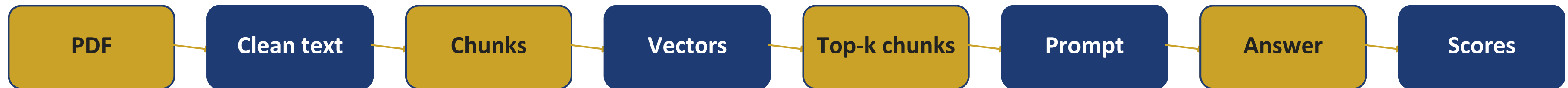
5. Answer generation

- Build fixed prompt and context; gemma4:31b-cloud answers

6. Evaluation & logging

- RAGAS scores plus timestamps per stage

Module Workflow Explanation



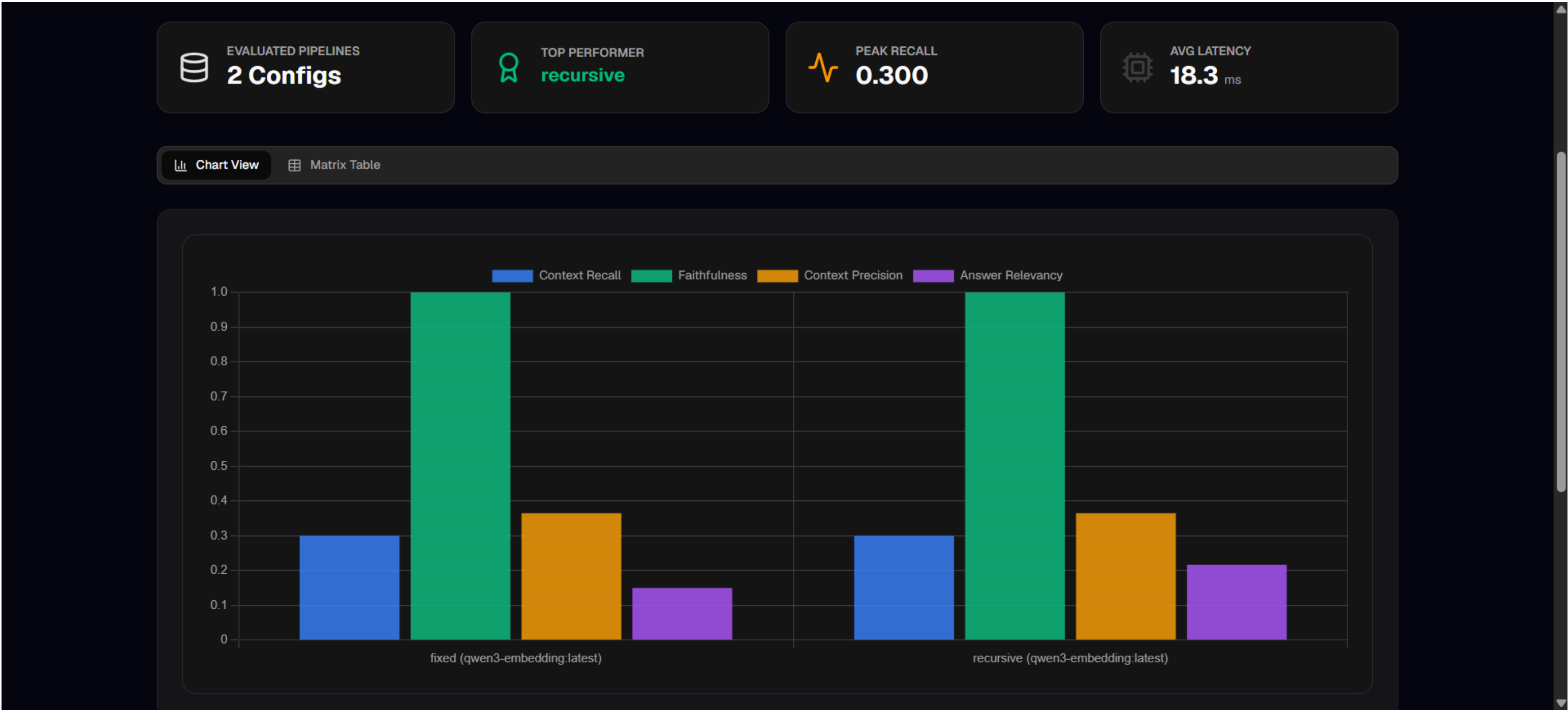
- **Step 1** ingestion normalizes the PDF into one text stream shared by all chunkers
- **Step 2** the active config splits text (fixed window, delimiters, similarity shifts, token budget, or LLM proposition)
- **Step 3** each chunk is embedded and stored with source, offsets and chunk ID
- **Step 4** each question is embedded and matched by cosine similarity to get top-k context
- **Step 5** context and question fill a constant prompt; gemma4:31b-cloud writes the answer
- **Step 6** contexts, answers and timings are logged; RAGAS scores them per configuration

Implementation and Coding

```
# Core pipeline (simplified pseudocode)
cfg = load_config(chunker, k, temp, top_p)
text = normalize(read_pdf(path))
chunks = CHUNKERS[cfg.chunker](text)
vecs = embed("qwen3-embedding", chunks)
index.add(vecs, chunks, metadata)
for q in questions:
    ctx = index.search(embed(q), k=cfg.k)
    ans = llm("gemma4:31b-cloud",
              prompt(q, ctx))
    log(ctx, ans, timings)
scores = ragas.evaluate(logs)
```

- **Design** one module per stage, driven by a config manifest
- **Constant** prompt, temperature, top-p, questions, embedding model
- **Chunkers** fixed, recursive, semantic, token, agentic
- **Logged** retrieved contexts, answers, stage timestamps, RAGAS scores
- **Note** replace with your actual code or screenshots

Snapshots of the Project



Testing

Test setup

- Corpus: evaluation PDF (~350 KB)
- 5 configurations C01–C05 on the same questions
- Same prompt, temperature, top-p and top-k
- Latency timed with identical boundaries
- Outputs stored for inspection

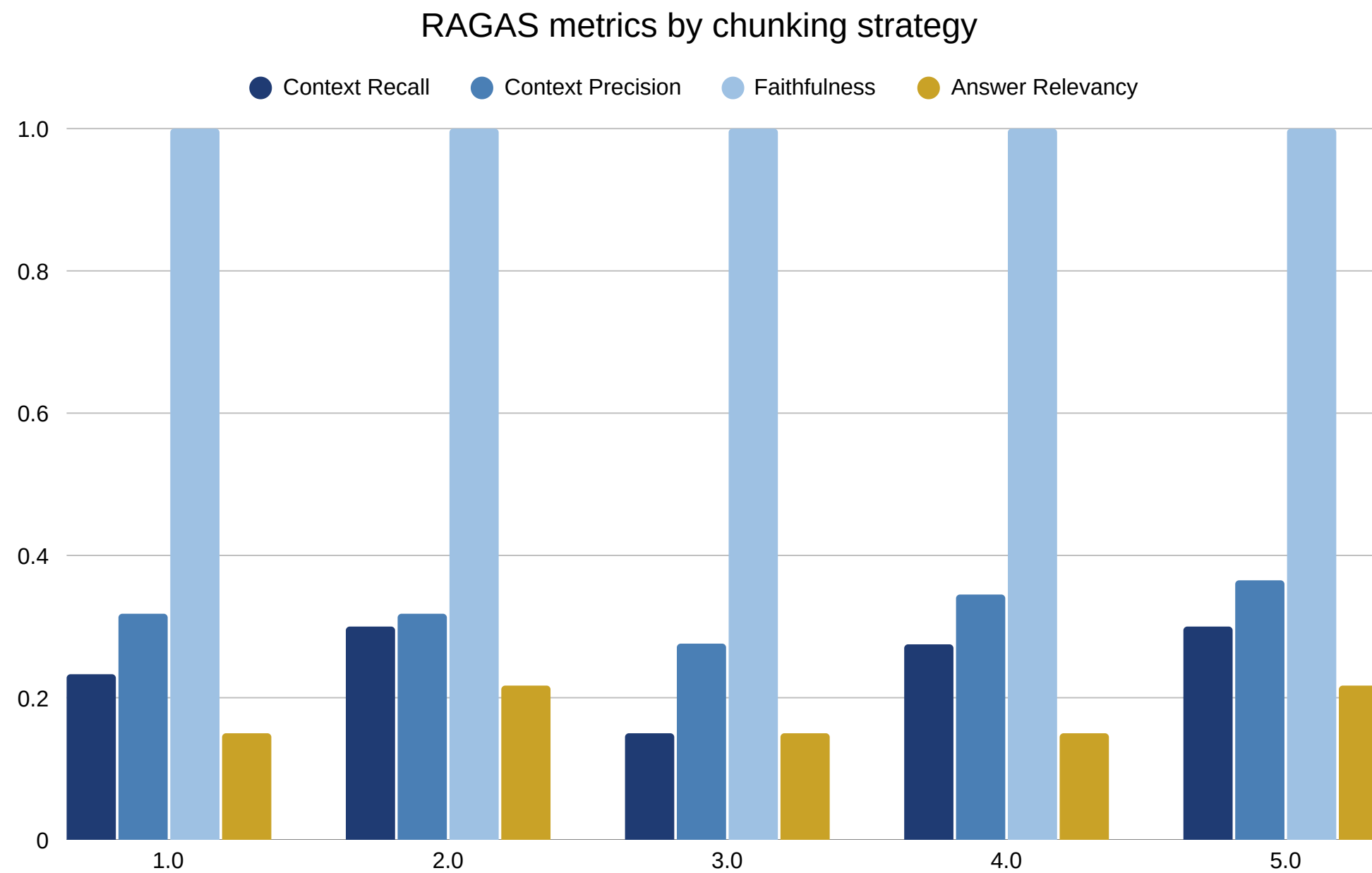
Metrics captured

- Context Recall, Context Precision
- Faithfulness, Answer Relevancy
- Latency: indexing, retrieval, generation, total
- Composite score $S(c)$ and tie-breaks

Each configuration uses qwen3-embedding:latest + gemma4:31b-cloud. Results come from single runs.

Result and Discussion: Input and Output

Input: PDF corpus + question set + chunking config **Output:** answer, retrieved context, scores, latency



Values are single-run benchmark figures; replace with final experiment logs if they differ.

Result Discussion

- **Agentic (C05)** best $S(c) = 0.555$ and highest Precision (0.365) at 7.28 s
- **Fixed (C02)** $S(c) = 0.539$ at 4.90 s, the best speed–quality balance
- **Semantic (C03)** fastest (3.54 s) but weakest retrieval (Recall 0.150)
- **Token (C04)** strong Precision (0.345) but slowest at 11.82 s
- **Faithfulness is 1.000 everywhere** differences come from retrieval, not generation
- **Pareto frontier** C03, C02 and C05; C01 and C04 are dominated

Conclusion

- **Summary** treated RAG as modular components and isolated the effect of chunking on retrieval, generation and latency
- **Findings** agentic chunking led on quality; fixed-size chunking gave the best efficiency; semantic chunking was fastest but least effective here
- **Takeaway** the best configuration depends on workload and latency budget
- **Future work** reranking and hybrid retrieval, hyperparameter grid search, more domains, human evaluation and repeated trials